

Alfredo's Pizzeria

Overview

I created this project to mimic the needs and challenges of a local food business that deals in take-outs and deliveries.

"Alfredo's" is the fictional name I attributed to the business; I purchased the dataset as it was not readily available. It was created from an actual **pizza delivery business**, but the names and addresses were replaced and randomised to protect names and likenesses.

The dataset consisted of three Excel files containing information about the orders delivered, stock and inventory, and staff information.

Research Questions

As there were three books containing different kinds of information, several questions could be asked. I went through each of the books and penned down the following as potential needs that such a business would require:

1. Orders Activity

- Revenue by pizza type
- Orders/sales by hour
- Orders by location
- Total orders and sales
- Quantity of items sold

2. Stock control/ Inventory management

- When to order new stock?
- What ingredients go into making each menu item?
- How much does it cost to make each menu item?
- Existing stock level?

3. Staff Management

- Work times for each staff
- Pay-per-hour
- Total amount of wages being paid
- The total amount of working hours

Data Wrangling

I began by normalising the data. I broke down each of the three main tables (Orders, Stock and Staff) into smaller tables, with their appropriate data grouped.

Given I was going to query the data via SQL, I had to visualise the schema and assign appropriate primary and foreign keys. Below is the schema I used while normalising the data.

ORDERS

- Item name
- Item Price
- Quantity
- Customer name
- Delivery address

CUSTOMERS

- **Cust_id**
- First_name
- Last_name

ADDRESS

- **Add_id**
- Del_ac
- Del_ac
- Del_ci

Data Querying

Given that I was working locally, I had to manually create each table in the Postgresql database and then import the data within the table from the CSV files, which I had converted from Excel format.

```
C: > Users > s.karunwi > Documents > pizzeria.sql
1  -----
2  --Setting up the tables--
3  -----
4
5  CREATE TABLE
6      orders (
7      row_id int PRIMARY KEY NOT NULL,
8      order_id varchar(10) NOT NULL,
9      created_at timestamp NOT NULL,
10     item_id varchar(10) NOT NULL,
11     quantity int NOT NULL,
12     cust_id int NOT NULL,
13     delivery boolean NOT NULL,
14     add_id int NOT NULL
15 );
16
17 CREATE TABLE
18     customers (
19     cust_id int PRIMARY KEY NOT NULL,
20     cust_firstname varchar(50) NOT NULL,
21     cust_lastname varchar(50) NOT NULL
22 );
23
```

Next, I began writing the queries to extract the needed information based on my business questions.

The first query was to connect the orders, items and address table. The resulting table provided data from which I could easily extract the significant metrics that would form the dashboard for Order activity.

```

1  SELECT
2      o.order_id,
3      i.item_price,
4      o.quantity,
5      i.item_category,
6      i.item_name,
7      o.created_at,
8      a.delivery_address1,
9      a.delivery_address2,
10     a.delivery_city,
11     a.delivery_zipcode,
12     o.delivery
13 FROM orders o
14 LEFT JOIN items i
15     ON o.item_id = i.item_id
16 LEFT JOIN address a
17     ON o.add_id = a.add_id;

```

Data Output Messages Notifications

	order_id character varying (10)	item_price numeric (10,2)	quantity integer	item_category character varying (100)	item_name character varying (100)
1	109	12.00	2	Pizza	Pizza Margherita Reg
2	110	16.00	1	Pizza	Pizza Diavola (hot) Reg
3	111	12.00	1	Pizza	Pizza Margherita Reg

Satisfied with the results, I began querying for the inventory management dashboard. Looking through the schema, I knew I had to use a subquery(s) to get what I wanted.

I wrote my first query for the number of orders per available item on the menu. I did this by connecting the orders table to the items table.

Next, I broke down each item ordered by the ingredients that went into preparing it. Three tables were connected to this query: Items, Recipes, and Ingredients.

The primary metrics were set in the resulting table; however, I needed to add calculated columns based on the business questions. With the table mentioned above as a subquery, I added calculated columns describing the ordered weight of each item, the unit cost of each ingredient, and the total ingredient cost. The resulting table solved most of the business questions about stock control.

```

1  ✓ SELECT
2      s1.item_name,
3      s1.ing_id,
4      s1.ing_name,
5      s1.ing_price,
6      s1.ing_weight,
7      s1.order_quantity,
8      s1.recipe_quantity,
9      (s1.recipe_quantity * s1.order_quantity) AS ordered_weight,
10     ROUND((s1.ing_price/s1.ing_weight),6) AS unit_cost,
11     ROUND((s1.recipe_quantity * s1.order_quantity)*(s1.ing_price/s1.ing_weight)
12 FROM (
13     SELECT
14         o.item_id,
15         i.sku,
16         i.item_name,
17         ing.ing_name,
18         r.ing_id,
19         ing.ing_weight,
20         ing.ing_price,
21         r.quantity AS recipe_quantity,
22         sum(o.quantity) AS order_quantity
23     FROM orders o
24     LEFT JOIN items i
25         ON o.item_id = i.item_id
26     LEFT JOIN recipes r
27         ON i.sku = r.recipe_id
28     LEFT JOIN ingredients ing
29         ON r.ing_id = ing.ing_id
30     GROUP BY (
31         o.item_id

```

Data Output Messages Notifications

	item_name character varying (100)	ing_id character varying (10)	ing_name character varying (200)	ing_price numeric (5,2)	ing_weight integer
1	Pizza Napolitana Large	ING001	Pizza dough ball (8 pack)	4.22	200
2	Pizza Permiana Large	ING001	Pizza dough ball (8 pack)	4.22	200

I still needed data for the remaining stock amount based on the total weight ordered and prior inventory amount. I saved the earlier table as a view and queried off of it.

I queried for the ordered weight and used this as a subquery to join with the Inventory table. From the resulting table, I queried for the total inventory weight before sales began and the weight left after sales were made in the period.

```

1  -- Joining the inventory table to get the quantity for each ingredient in stock
2  ✓ SELECT *
3  FROM(
4      SELECT
5          ing_id,
6          ing_name,
7          sum(ordered_weight) AS ordered_weight
8      FROM inv_mgt
9      GROUP BY ing_name,ing_id
10 ) s2
11 LEFT JOIN inventory inv
12     ON s2.ing_id = inv.item_id;
13
14 -- Querying for the total inventory weight
15 ✓ SELECT
16     s2.ing_name,
17     s2.weight_ordered,
18     (ing.ing_weight*inv.quantity) AS total_inv_weight,
19     (ing.ing_weight*inv.quantity) - s2.weight_ordered AS weight_left_in_stock
20 FROM(
21     SELECT
22         ing_id,
23         ing_name,
24         sum(ordered_weight) AS weight_ordered
25     FROM inv_mgt
26     GROUP BY ing_name,ing_id
27 ) s2
28 LEFT JOIN inventory inv
29     ON s2.ing_id = inv.item_id
30 LEFT JOIN ingredients ing
31     ON s2.ing_id = ing.ing_id;

```

Data Output Messages Notifications



	ing_name character varying (200)	weight_ordered numeric	total_inv_weight integer	weight_left_in_stock numeric
1	Pizza dough ball (8 pack)	78650	100000	21350
2	Tomato sauce	20600	112500	91900

The data in the resulting table was sufficient to answer the remaining questions about Inventory management.

The last table required was to answer the questions relating to staff management.

It was straightforward as I needed to write a query to connect the three tables with staff information: Staff, Rotations and Shifts. Within the query, I added two calculated columns for hours worked (hours_in_shift) and wage bill for each staff based on hours worked (staff_cost).

Query Query History

```
1  SELECT
2      ro.shift_id,
3      ro.staff_id,
4      sh.shift_id,
5      st.staff_firstname,
6      st.staff_lastname,
7      ro.date,
8      st.hourly_rate,
9      sh.start_time,
10     sh.end_time,
11     ROUND((EXTRACT (EPOCH FROM(sh.end_time - sh.start_time)))/3600,2) AS hours
12     ROUND((EXTRACT (EPOCH FROM(sh.end_time - sh.start_time)))/3600,2) * st.hourly_rate AS total_pay
13 FROM staff st
14 LEFT JOIN rotations ro
15     ON st.staff_id = ro.staff_id
16 LEFT JOIN shifts sh
17     ON ro.shift_id = sh.shift_id;
```

Data Output Messages Notifications

	shift_id character varying (20)	staff_id character varying (20)	shift_id character varying (20)	staff_firstname character varying (50)	staff_lastname character varying (50)
1	sh0005	st0001	sh0005	Mindy	Sloan
2	sh0005	st0002	sh0005	Luqman	Cantu
3	sh0005	st0009	sh0005	Lilly-Rose	Vaughn
4	sh0005	st0010	sh0005	Desiree	Gardner
5	sh0006	st0001	sh0006	Mindy	Sloan
6	sh0006	st0002	sh0006	Luqman	Cantu

Data Visualization & Insights

The queries were done, and I had the tables I needed to create a dashboard for business stakeholders. I connected PostgreSQL to Power BI to develop the needed dashboards.

I extracted the four tables I needed to complete the dashboard: Orders activity, Stock 1&2, and Staff cost. I made a few rectifications to clean up each table, such as changing the column names, ensuring each column had the correct data type, splitting columns to normalise the data, etc.

Queries [4]

- Orders_activity
- stock 1
- stck 2
- Staff Cost (1)

✕

✓

fx

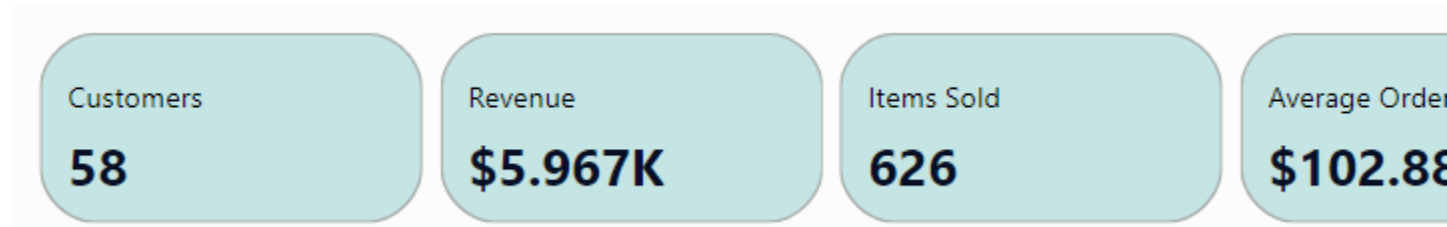
= Table.RenameColumns("#"Changed Type2",{

	123 order_id	123 cust_id	123 add
	Valid Error Empty 100% 0% 0%	Valid Error Empty 100% 0% 0%	Valid Error Empty 100% 0% 0%
1	109	1	
2	110	2	
3	111	3	
4	111	3	
5	112	4	
6	112	4	
7	112	4	
8	112	4	
9	112	4	
10	113	5	
11	113	5	
12	113	5	
13	113	5	
14	113	5	
15	114	6	
16	114	6	
17	114	6	
18	114	6	
19	114	6	
20	114	6	
21	114	6	

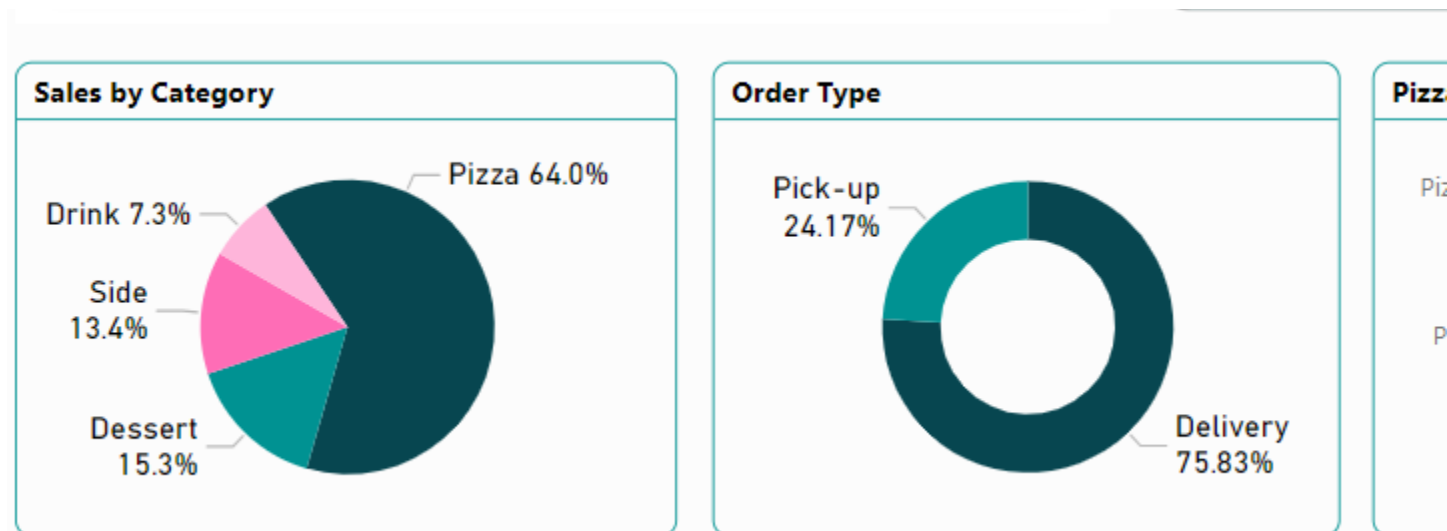
Most of the heavy lifting had been done in PostgreSQL, and the dashboards didn't take too long to set up. I created four measures to help with the aggregation: Average order value, Total sales, Total hours worked, and Total staff cost.

Dashboard 1 - Orders Activity

The first visuals were cards conveying total aggregations within the period. Number of customers serviced, Revenue generated, items sold and Average value per order.



Next, I created a pie chart to show the sales by category (pie chart because it's a pizzeria, and I was highlighting pizza sales). The doughnut chart was used for the percentage per order type (doughnut chart to reduce the data-ink ratio), and finally, a bar chart to show the sales made per pizza item on the menu. I made the following inferences:

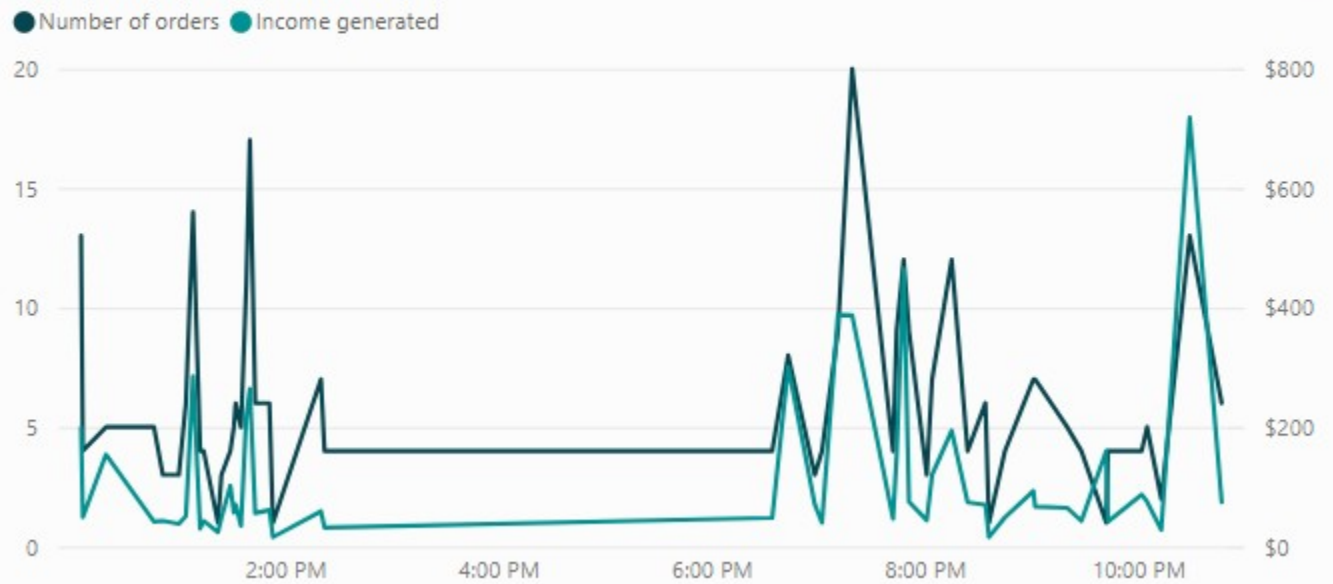


- Unsurprisingly, Pizza's were bringing in over 60% of the revenue
- Customers ordered more desserts than sides
- The Pizza Quattro Formaggi Large was the top-selling pizza with over \$200 worth of sales over the second favourite - Pizza Diavola (hot) Large.

The fifth visual was a line graph to display the number of orders against the number of sales made for every working hour within the period. The last visual was a map showing the location of each order that was made. I was able to infer the following:

Activity by Hour

Orders & Sales



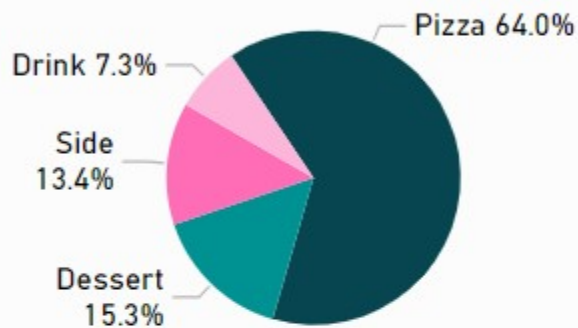
- The working hours were between 12:00 to 23:00
- Peak working hours were 13:00, 19:00, and 22:00
- All orders came from the Manchester area in Connecticut, United States
- Most of the bulk orders came outside the Manchester centre

Alfredo's Orders Activity

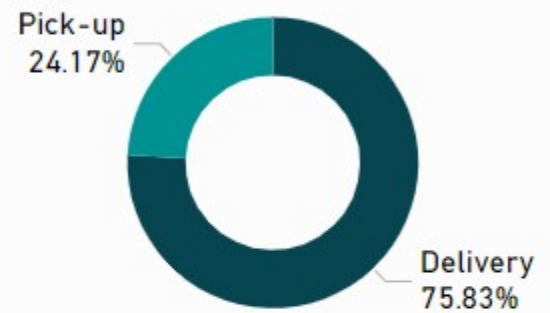
Customers

58

Sales by Category

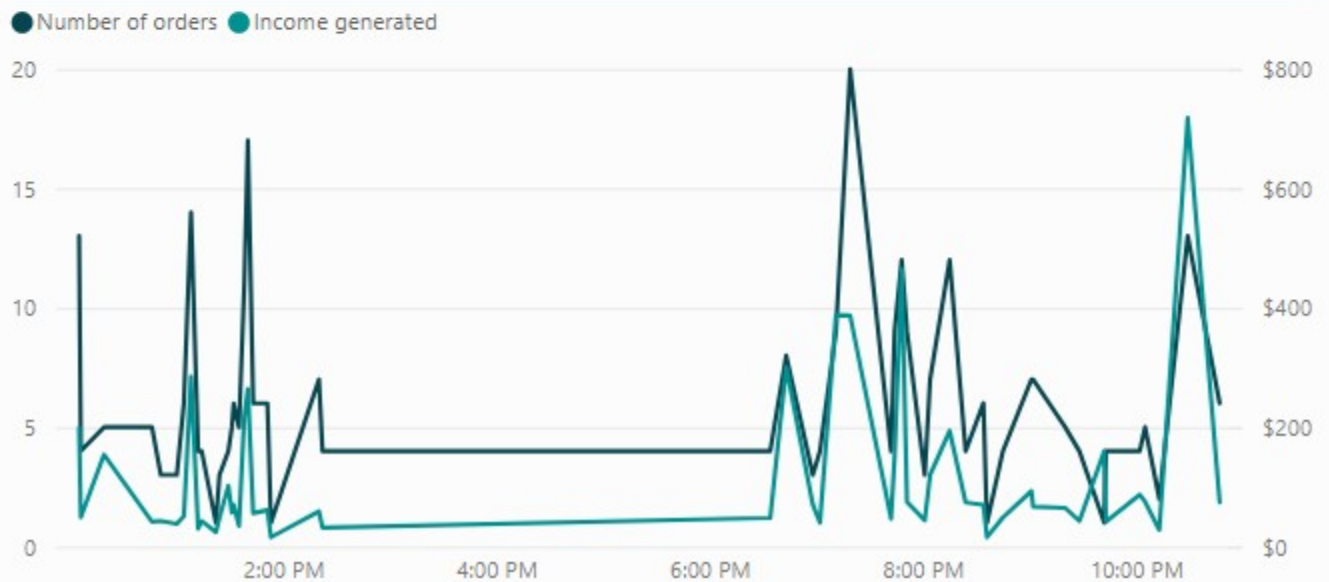


Order Type



Activity by Hour

Orders & Sales



Dashboard 2 - Inventory Management

The first visual was a card which displayed the total amount it cost to make all orders within the period regarding ingredients.

The second visual was a table displaying each ingredient by name, the quantity used to prepare orders, the total cost of each ingredient used, and the percentage left in stock. I used a colour code to quickly identify the ingredients that needed replacement - red (<30% left in stock), pink (<50% left in stock, and no colour (>50% left in stock).

The third visual was a multi-row card with every item on the menu and the amount it costs to prepare. Clicking on each item modified the ingredient table to show only the ingredients required to make the highlighted item. I inferred the following:

Alfredo's Inventory Management

Ingredient	Quantity Ordered	Total Ingredient Cost	Remaining Stock
Anchovies	1,850	\$20.34	7.5
Pizza dough ball (8 pack)	78,650	\$166.00	21.3
Ricotta cheese	10,320	\$27.45	31.2
Chocolate brownie	3,225	\$5.74	35.5
Parmesan cheese	13,850	\$103.88	44.6
Spicy salami	3,390	\$36.46	51.5
Mozzarella cheese	42,300	\$244.50	57.7
Chocolate ice cream	3,700	\$12.70	58.8
Croutons	1,850	\$7.77	70.4
Gorgonzola cheese	10,320	\$81.50	70.5
Fruit salad	2,850	\$4.06	71.5
Pepperoni	6,320	\$61.12	74.7
Fanta Regular 33cl	29	\$11.89	75.8
Tuna	1,250	\$4.79	79.1
Tomato sauce	20,600	\$17.79	81.6
Rotisserie chicken pieces	4,440	\$30.59	82.2

- Anchovies and the Pizza dough ball (8 pack) needed immediate restocking.
- The Pizza Seafood Large was the most expensive item to make on the menu.

- Although the Pizza dough ball was used the most, the mozzarella cheese took the most chunk out of the business pockets; it accounted for almost 25% of the total ingredients cost.

Dashboard 3 - Staff Management

The first visual was two cards that displayed the aggregation for the total wage bill for the period and the total hours worked for the field staff.

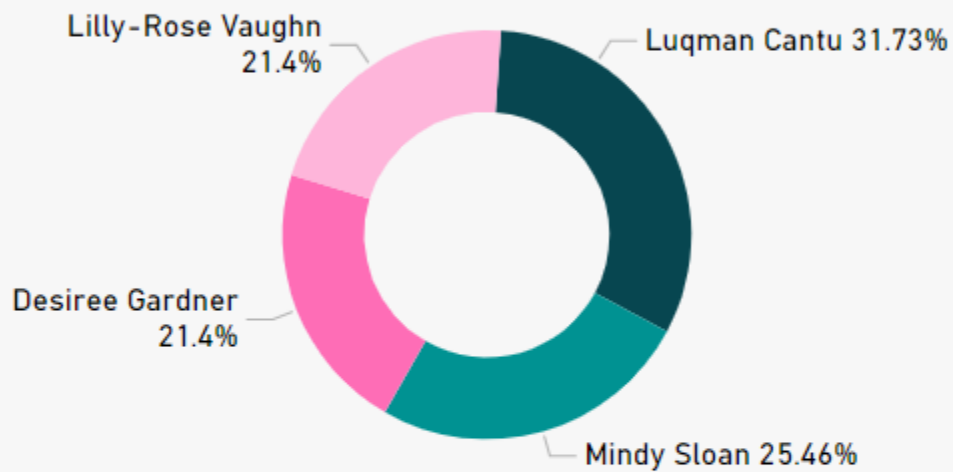
The second visual was a multi-card row showing the working hours of each staff for the period.

The third visual was two doughnut charts that showed the wage bill per field staff member and the total number of hours worked.

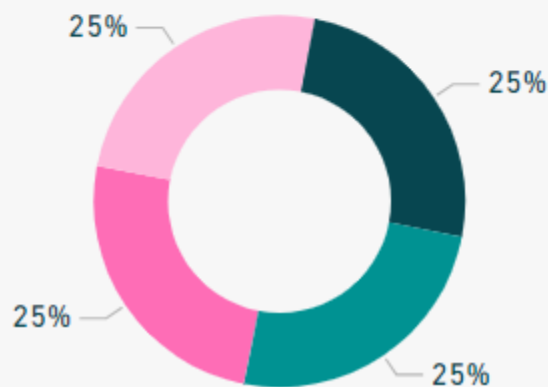
Alfredo's Staff Information

Hourly Rate

& total cost per staff



Hours Worked



Conclusions

I was able to draw the following conclusions based on the initial business questions:

1. The Pizza Seafood Large is the most expensive to make and only brought in \$40 from 2 customers within the period, second only to the Pizza Hawaiian Regular, the least-selling pizza item.
2. Most orders were recorded at 19:00, which is generally dinner time at the average home, 22:00, which could mean it was the last resort if people could not decide what to have for dinner, and 13:00, usually lunch break at most organisations.
3. Generally, small orders were recorded close to the city centre, and the more significant number of orders were recorded by individuals staying farther from the city centre.
4. The Pizza dough ball (8-pack) and Anchovies were being run through a lot quicker than any other ingredient and needed to be ordered in large quantities compared to other ingredients.
5. Most of the ingredients were still way above 50% in stock.
6. All the field staff were working two shifts and working the same hours. However, Luqman Cantu's pay rate was significantly higher than the rest of the staff members.